Image Processing

Lecture 4: Geometric Operations

By

Dr. Rasha Mahmoud Kamel

Introduction

- Geometric operations modify the geometry of an image by repositioning pixels in a constrained way.

(a)	(b)	(c)	(d)

Examples of typical geometric operations: **(a)** original image; **(b)** translation (shifting); **(c)** scaling; **(d)** rotation.

Introduction

- Geometric transformations of digital images consist of two basic operations:

  (1) Spatial transformation of coordinates that is performed using the mapping function.

  (2) Intensity interpolation that ***assigns intensity values to the spatially transformed pixels.*** Interpolation methods are used to compute the new value of each pixel in the spatially transformed image.

Affine Transformations

- **A geometric operation** can be described mathematically as ***the process of transforming an input image*** $I_{\text{Input}}(x, y)$ ***into a new image*** $I_{\text{Output}}(x', y')$ by modifying the coordinates of image pixels:

$$I_{\text{Input}}(x, y) \rightarrow I_{\text{Output}}(x', y')$$

- That is, the pixel value originally located at coordinates (x, y) will be relocated to coordinates (x', y') in the output image.

- To model this process, a mapping function is needed. The **mapping function** ***specifies the new coordinates (in the output image) for each pixel in the input image:***

$$(x', y') = \boldsymbol{T}(x, y)$$

Affine Transformations

- This **mapping function** is an arbitrary **2D *function***. It is often specified as two separate functions, one for each dimension:

$$x' = \boldsymbol{T_x}(x, y)$$
$$y' = \boldsymbol{T_y}(x, y)$$

where $\boldsymbol{T_x}$ and $\boldsymbol{T_y}$ are usually expressed as polynomials in x and y.

- The case where $\boldsymbol{T_x}$ and $\boldsymbol{T_y}$ are linear combinations of x and y is called ***affine transformation*** (or ***affine mapping***):

$$x' = a_0 x + a_1 y + a_2$$
$$y' = b_0 x + b_1 y + b_2 \qquad (*)$$

Affine Transformations

- Equations (*) can also be expressed in matrix form as follows:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \qquad (**)$$

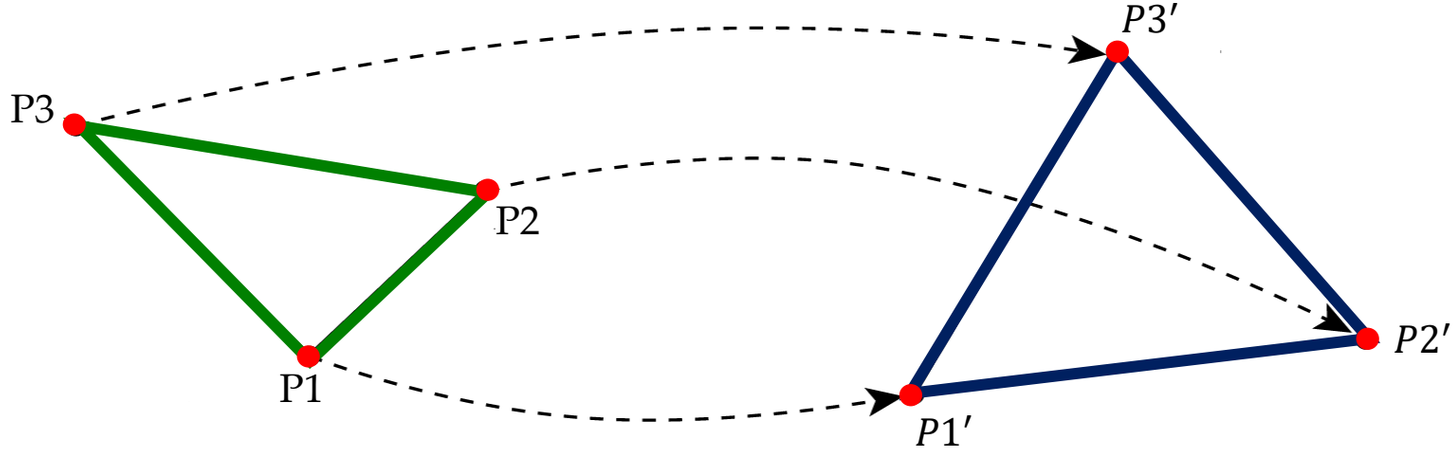
  where $a, b, c, d, e,$ and f are the 2D ***affine mapping parameters*** that are uniquely determined by ***three pairs of corresponding points.***

- ***Given the coordinates of relevant points before and after the transformation***, one can write n equations in x and y and solve them to find the n transformation coefficients.

Affine Transformations

- **Affine mapping** transforms ***straight lines to straight lines***, ***triangles to triangles***, and ***rectangles may become parallelograms***. Parallel lines remain parallel and the distance ratio between points on a straight line does not change.

- ***The most common geometric operations are*** translation, scaling, rotation, and shearing. These transformations are performed using the unified representation in Equation (**).



Mapping one triangle onto another by an affine transformation

Affine Transformations

Translation	$\boldsymbol{P}' = \boldsymbol{T} \times \boldsymbol{P}$	$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$	t_x and t_y are translation factors
Rotation	$\boldsymbol{P}' = \boldsymbol{T} \times \boldsymbol{P}$	$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$	θ is the rotation angle (Counterclockwise)
Scaling	$\boldsymbol{P}' = \boldsymbol{T} \times \boldsymbol{P}$	$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$	s_x and s_y are scaling factors
Shearing	$\boldsymbol{P}' = \boldsymbol{T} \times \boldsymbol{P}$	$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & sh_x & 0 \\ sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$	sh_x and sh_y are shearing factors

Composite Affine Transformations

- A composite affine transformation means combining multiple affine operations—like translation, rotation, scaling, and shearing—into a single transformation matrix. This is done by matrix multiplication of individual transformation matrices in a specific order.

- **For example,** if we want to resize an image with scaling matrix $\boldsymbol{S}$, rotate it with rotation matrix $\boldsymbol{R}$, and move the result to some location with translation matrix $\boldsymbol{N}$, we simply form a 3 × 3 matrix equal to the product of the scaling, rotation, and translation matrices as follows:

- The composite representation for scaling, rotation, and translation (in that order):

$$\boldsymbol{T} = \boldsymbol{N} \times \boldsymbol{R} \times \boldsymbol{S}$$

Composite Affine Transformations

- The composite representation for scaling, rotation, and translation (in that order):

$$\boldsymbol{T} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- Thus, each pixel (x, y) in the input image is mapped to a new pixel (x', y') in the output image:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- The inverse transform is

$$\boldsymbol{T}^{-1} = \boldsymbol{S}^{-1} \times \boldsymbol{R}^{-1} \times \boldsymbol{N}^{-1}$$

Inverse of Affine Transformations

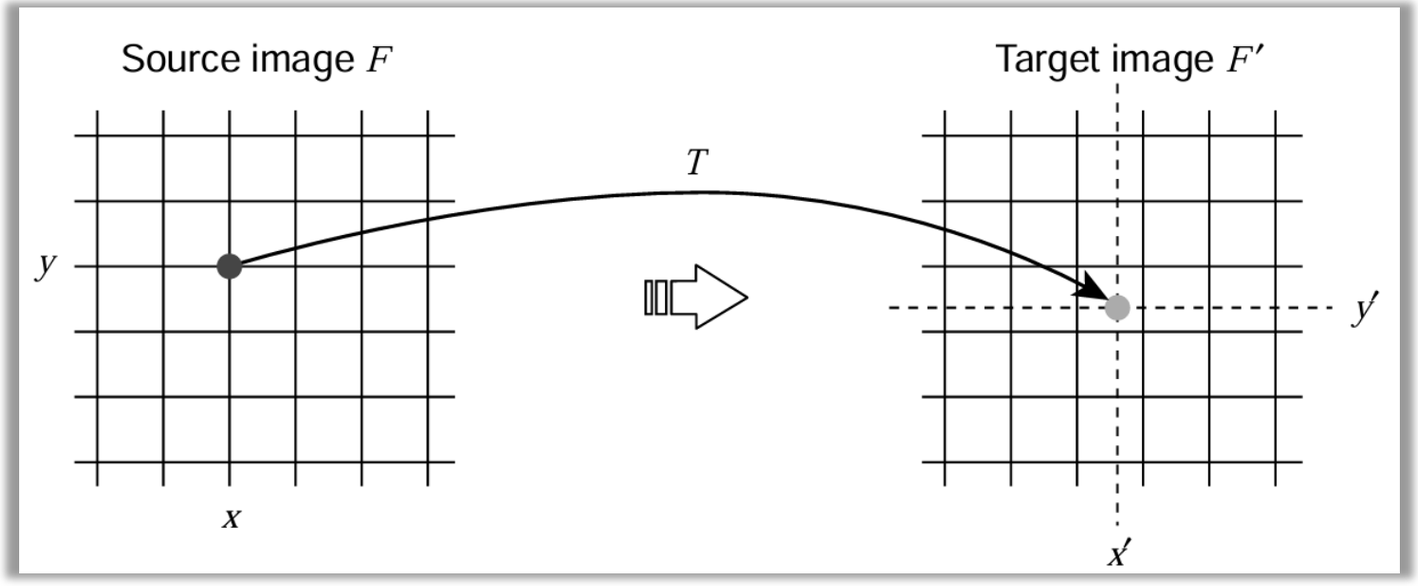
Translation	$\boldsymbol{P} = \boldsymbol{T}^{-1} \times \boldsymbol{P}'$	$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & -t_x \\ 0 & 1 & -t_y \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$	t_x and t_y are translation factors
Rotation	$\boldsymbol{P} = \boldsymbol{T}^{-1} \times \boldsymbol{P}'$	$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$	θ is the rotation angle (Clockwise)
Scaling	$\boldsymbol{P} = \boldsymbol{T}^{-1} \times \boldsymbol{P}'$	$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1/s_x & 0 & 0 \\ 0 & 1/s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$	s_x and s_y are scaling factors
Shearing	$\boldsymbol{P} = \boldsymbol{T}^{-1} \times \boldsymbol{P}'$	$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} -1/(sh_x * sh_y - 1) & sh_x/(sh_x * sh_y - 1) & 0 \\ sh_y/(sh_x * sh_y - 1) & -1/(sh_x * sh_y - 1) & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$	

Interpolation Methods

- ***After a geometric operation has been performed*** on the original image, the resulting value for each pixel can be computed in two different ways:
  - The first one is called **forward mapping**—also known as **source-to-target mapping**—approach.
  - The second one is called **backward mapping**—also known as **target-to-source mapping**—approach.

Interpolation Methods – Forward Mapping

- **Forward mapping** approach consists of iterating over every pixel of the input image, computing its new coordinates, and copying the value to the new location.



For each pixel position in the input image, the corresponding target position—resulting from applying a geometric transformation $\boldsymbol{T}$—is found in the output image. The target position (x', y') does not coincide with any discrete raster point, and the value of the pixel (x, y) in the input image is copied to one of the adjacent target pixels.

Interpolation Methods – Forward Mapping

- However, ***this approach has a number of problems***, such as the following:

  (1) ***Many coordinates*** calculated by the transformation equations ***are not integers***, and need to be rounded off to the closest integer to properly index a pixel in the output image.

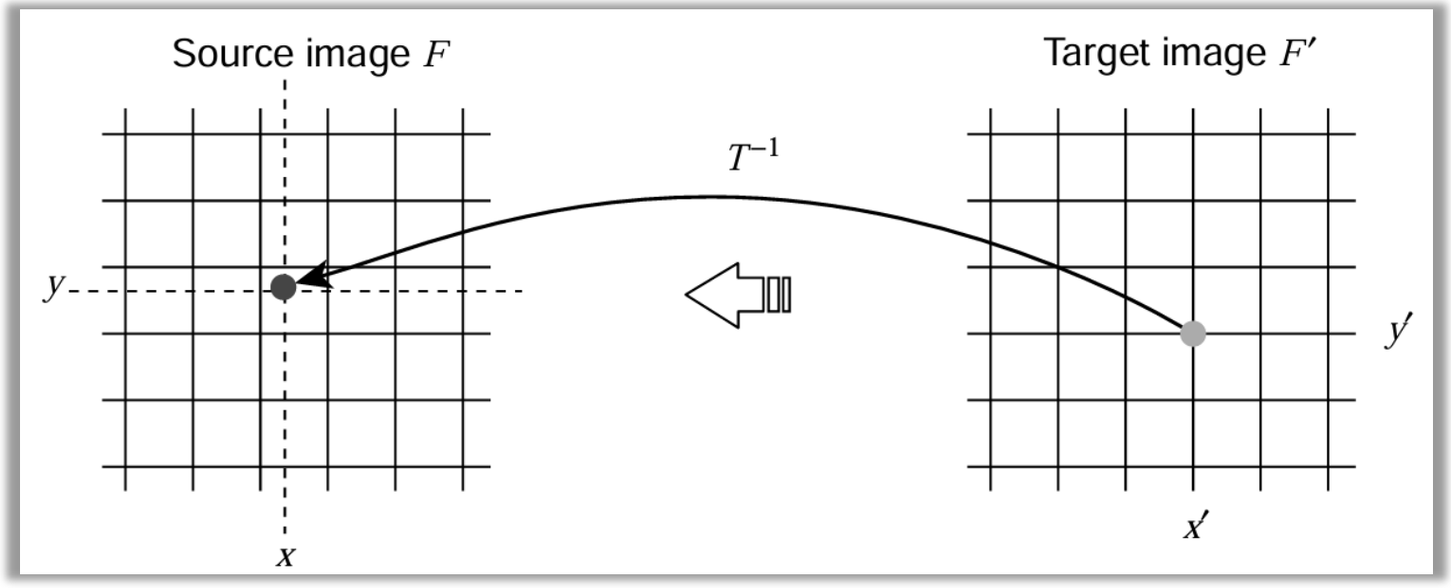
  (2) ***Many coordinates*** may lie out of bounds (e.g., negative values).

  (3) ***Many output pixels' coordinates are addressed several times during the calculations*** (which is wasteful) ***and some are not addressed at all*** (which leads to "holes" in the output image, meaning that no pixel value was computed for that coordinate pair).

Interpolation Methods – Backward Mapping

- The solution to these limitations usually comes in the form of a **backward mapping** approach.
- This approach consists of ***visiting every pixel in the output image and applying the inverse transformation*** to determine the coordinates in the input image from which a pixel value must be sampled.
- **Backward mapping** process usually requires some type of interpolation to compute the best value for every pixel in the output image.

Interpolation Methods – Backward Mapping



For each discrete pixel position (x', y') in the output image, the corresponding position in the input image (x, y) is found by applying the inverse mapping function $\boldsymbol{T}^{-1}$. The new pixel value is found by interpolation among the neighbors of (x, y) in the input image.

Simple Approach to Interpolation

- ***If we want to enlarge or reduce an image by a certain factor*** (e.g., a factor of 2 in both directions),

  (1) In the case of shrinking the image, we would probably deal with the problem of removing pixels by skipping every other pixel along each row and column.

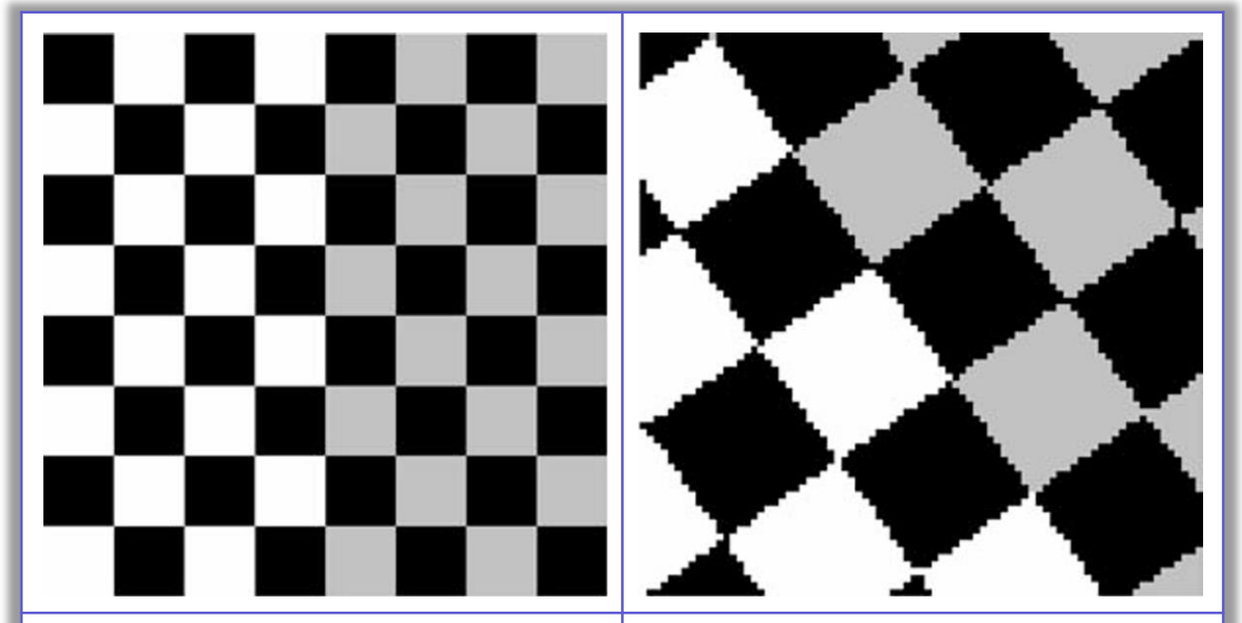
  (2) In the case of enlarging the image, we would probably copy each original pixel to an $n \times n$ block in the output image.

Simple Approach to Interpolation

- The interpolation schemes (pixel removal and pixel duplication) are fast and easy to understand, but **suffer from several limitations**, such as the following:
  - ***The "blockiness" effect*** that may become noticeable when enlarging an image, especially with large scaling factors.
  - ***The possibility of removing essential information*** in the process of shrinking an image.
  - ***The difficulty in extending*** these approaches ***to noninteger scaling factors*** (e.g. 1.5x, 2.3x).
- Other simplistic methods—such as ***using the mean (or median) value of the original*** $n \times n$ ***block in the input image*** to determine the value of each output pixel in the shrunk image—also produce low-quality results and are bound to fail in some cases. ***These limitations call for improved interpolation methods.***

Nearest-Neighbor Interpolation

- This scheme ***rounds off the calculated coordinates*** (x', y') ***to their nearest integers.***
- Zero-order (or nearest-neighbor) interpolation ***is simple and computationally fast.***
- However, it ***produces low-quality results*** with artifacts such as ***blockiness effects which are more pronounced at large-scale factors*** and ***jagged straight lines*** — particularly after rotations by angles that are not multiples of 90°.



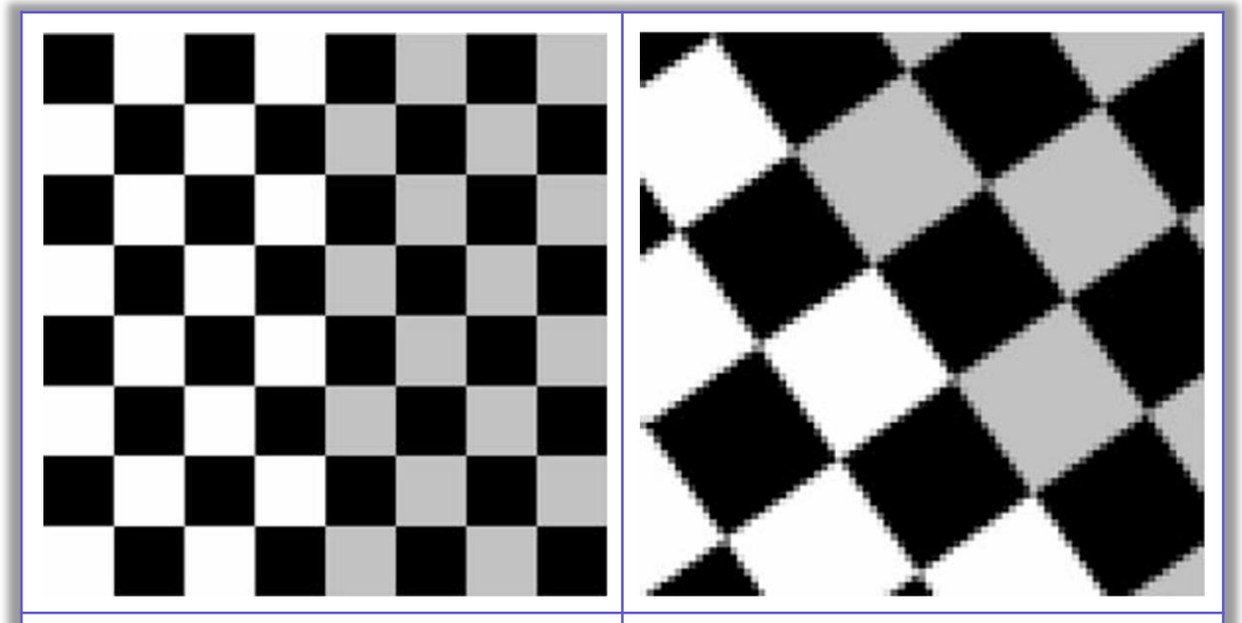
(a) **(b)**

(a) original image; **(b)** zoomed-in version of rotated (35°) image using nearest-neighbor interpolation technique.

First-Order (Bilinear) Interpolation

- This interpolation technique ***calculates the gray value of the interpolated pixel at coordinates*** (x', y') ***as a weighted function of the gray values of the four pixels surrounding the reference pixel in the input image.***

- Bilinear interpolation ***produces visually better*** results than the nearest-neighbor interpolation at the expense of additional CPU time.



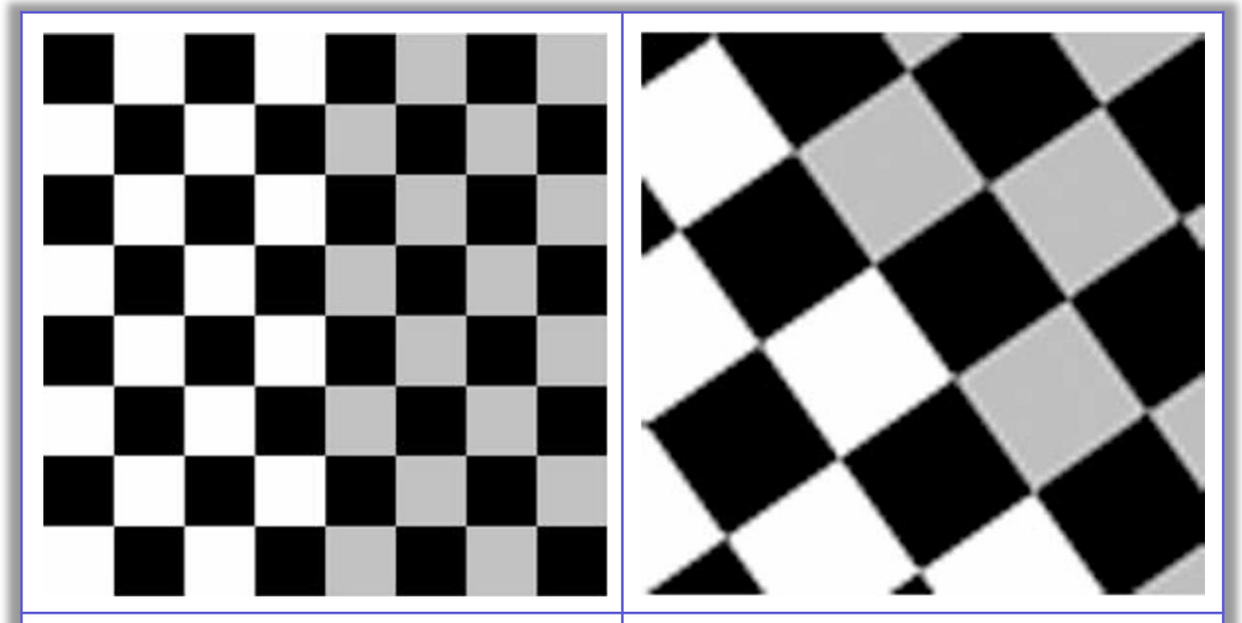
(a) **(b)**

(a) original image; **(b)** zoomed-in version of rotated (35°) image using Bilinear interpolation technique.

Higher Order Interpolation

- Higher order interpolations are more ***sophisticated and computationally expensive methods*** for interpolating the gray value of a pixel.

- The third-order interpolation scheme implemented in several **MATLAB** functions is also known as **bicubic interpolation**. ***It takes into account the 4×4 neighborhood around the reference pixel*** and computes the resulting gray level of the interpolated pixel by performing the convolution of the 4×4 neighborhood with a cubic function.



(a) **(b)**

(a) original image; **(b)** zoomed-in version of rotated (35°) image using Bicubic interpolation technique.

Affine Transformations – MATLAB

- The IPT has a built-in function called **imwrap** which is used to apply geometric transformation to image.

```
outputImage = imwarp ( inputImage , tform , interp)
```

- **tform** is an affine2d object that stores information about a 2D affine geometric transformation:

```
tform = affine2d ( T )
```

- $\boldsymbol{T}$ is a valid affine transformation matrix that must be of the form:

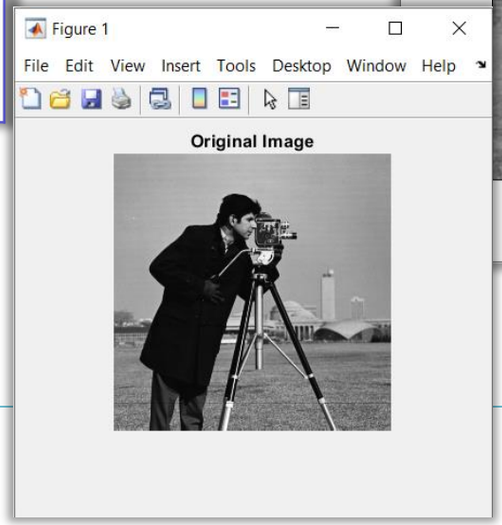
$$\boldsymbol{T} = \begin{bmatrix} a & d & 0 \\ b & e & 0 \\ c & f & 1 \end{bmatrix}$$

- **interp** specifies the type of interpolation to use (**'nearest'**, **'linear'** , **'cubic'**), where linear is the default.

Affine Transformations – MATLAB

- **Example:** Load the Cameraman image, and perform an affine transformation to scale the image by a factor of 2 in both the horizontal and vertical directions. Then, apply this transformation to the image and display the final result.

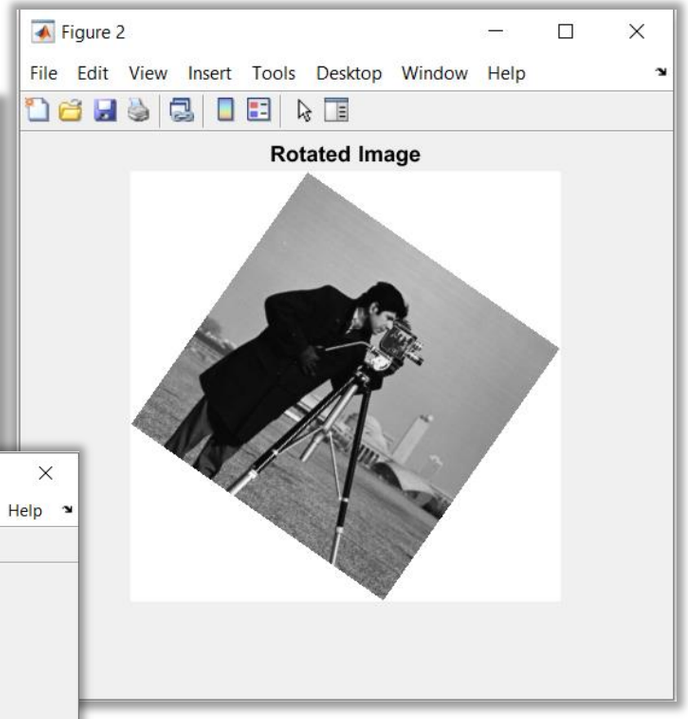
```
inputImage = imread('cameraman.tif');
sx = 2; sy = 2;
S = [sx 0 0; 0 sy 0; 0 0 1]';
tform = affine2d(S);
outputImage = imwarp(inputImage, tform);
figure, imshow(inputImage), title('Original Image')
figure, imshow(outputImage), title('Scaled Image')
```



Affine Transformations – MATLAB

- **Example:** Load the Cameraman image, and perform an affine transformation to rotate the image by an angle $\theta = 35^{\circ}$. Use white as the fill color to clearly visualize the rotation effect. Finally, apply this transformation to the image and display the final result.

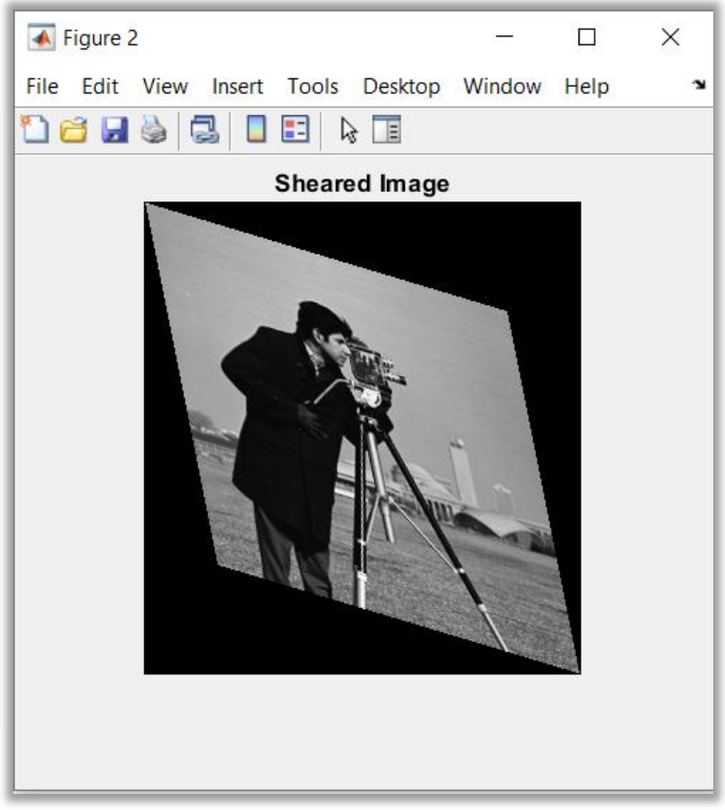
```
theta = 35;
R = [cosd(theta) -sind(theta) 0;sind(theta) cosd(theta) 0;0 0 1]';
tform = affine2d(R);
outputImage = imwarp(inputImage, tform, 'FillValues', 255);
figure, imshow(inputImage), title('Original Image')
figure, imshow(outputImage), title('Rotated Image')
```



Affine Transformations – MATLAB

- **Example:** Load the Cameraman image, and perform an affine transformation to shear the image by a factor [0.3 , 0.2] on the image. Then, apply this transformation to the image and display the final result.

```
inputImage = imread('cameraman.tif');
shx = 0.3; shy = 0.2;
H = [1 shx 0; shy 1 0;0 0 1]';
tform = affine2d(H);
outputImage = imwarp(inputImage, tform, 'cubic');
figure, imshow(inputImage), title('Original Image')
figure, imshow(outputImage), title('Sheared Image')
```



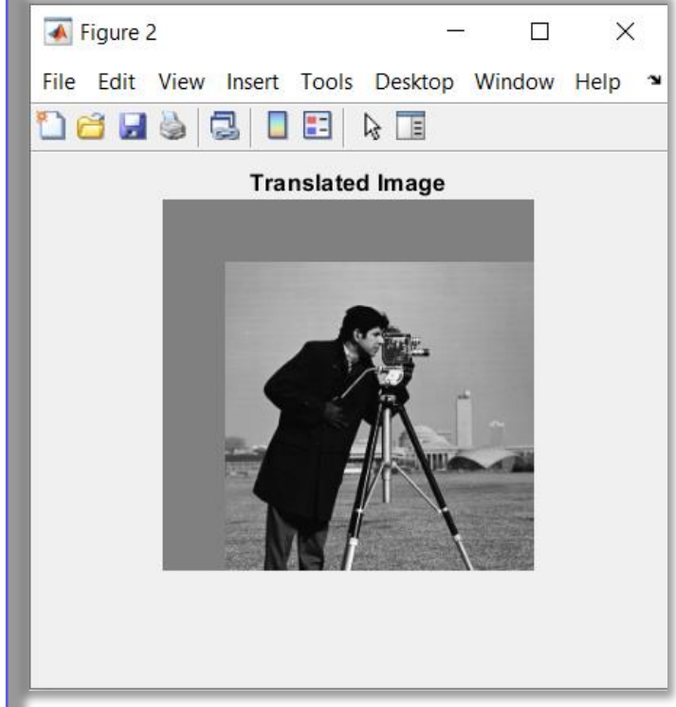
Affine Transformations – MATLAB

- **Example:** Load the Cameraman image, and perform an affine transformation to translate the image by a translation factor [50 , 50]. Use average gray as the fill color to clearly visualize the translation effect. Finally, apply this transformation to the image and display the final result.

```
inputImage = imread('cameraman.tif');
tx = 50; ty = 50;
T = [1 0 0; 0 1 0; tx ty 1];
tform = affine2d(T);

outputSize = [256, 256];
xWorldLimits = [0, outputSize(1) + 50];
yWorldLimits = [0, outputSize(1) + 50];
Rout = imref2d(outputSize, xWorldLimits, yWorldLimits);

outputImage = imwarp(inputImage, tform,'FillValues', 128, 'OutputView',Rout);
figure, imshow(inputImage), title('Original Image');
figure, imshow(outputImage,Rout), title('Translated Image');
```

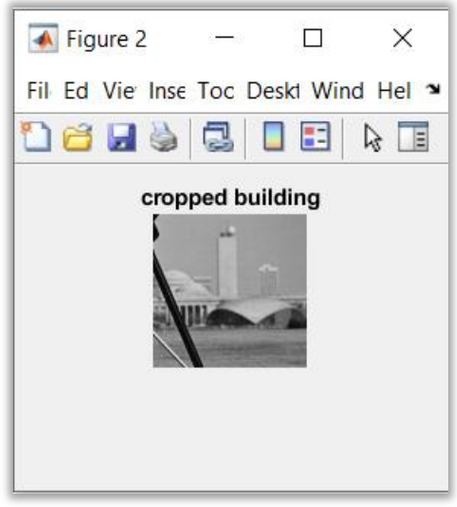


Other Geometric Operations – MATLAB

- **Cropping:** The IPT has a function for cropping images, **imcrop**, which crops an image to a specified rectangle. The coordinates of the croped rectangle are passed as parameters to the function.

outputImage = **imcrop** (inputImage , **[xmin ymin width height]**)

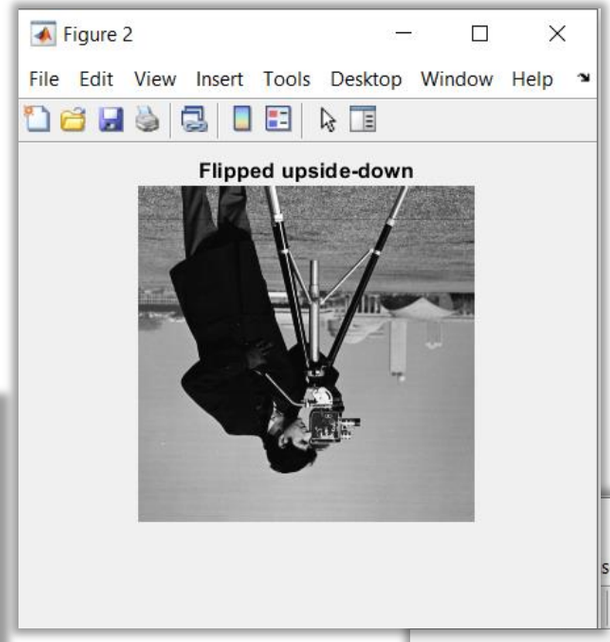
```
inputImage = imread('cameraman.tif');
xmin = 150; ymin = 100; width = 100; height = 100;
outputImage = imcrop(inputImage, [xmin ymin width height]);
figure, imshow(inputImage), title('Original image')
figure,imshow(outputImage), title('Cropped building')
```



Other Geometric Operations – MATLAB

- **Flipping:** The IPT has two functions for flipping images :
  - **flipud** which flips the image vertically (up to down).
  - **fliplr** which flips the image horizontally (left to right).

```
inputImage = imread('cameraman.tif');
imageUD = flipud(inputImage);
imageLR = fliplr(inputImage);
figure, imshow(inputImage), title('Original image')
figure, imshow(imageUD), title('Flipped upside-down')
figure, imshow(imageLR), title('Flipped left-right')
```



Thank You